

Chapter 43 — DP Optimization Patterns

0. What You Will Learn in This Chapter

DP Optimization is used when a normal DP recurrence is correct but too slow or too memory-heavy.

Many hard interview and competitive-style problems follow this pattern:

First derive the correct DP.
Then optimize it using structure.

This chapter is important for DE Shaw-level preparation because harder rounds may test whether you can move beyond a basic recurrence.

By the end of this chapter, you should understand:

- Why DP optimization is needed
- Space optimization
- Rolling array optimization
- Prefix sum optimization
- Difference-array optimization for DP transitions
- Monotonic queue optimization
- Binary search optimization inside DP
- LIS optimization
- Bitset optimization
- State compression
- Sparse memoization
- Divide and conquer DP optimization, conceptually
- Knuth optimization, conceptually
- Convex Hull Trick, conceptually
- How to recognize which optimization applies

This is a tough chapter, so explanations and examples will be more detailed.

Core problems covered in this chapter:

1. Fibonacci space optimization
2. Unique Paths rolling array
3. Minimum Path Sum rolling array
4. Range Sum Query style prefix optimization
5. Number of Ways to Split Array / prefix DP
6. Coin Change space optimization
7. 0/1 Knapsack rolling optimization
8. Longest Increasing Subsequence $O(n \log n)$
9. Maximum Sum Circular Subarray
10. Jump Game VI, monotonic deque DP
11. Constrained Subsequence Sum
12. Shortest Subarray With Sum at Least K, prefix + deque
13. Maximal Square space optimization
14. Count Vowels Permutation, state compression
15. Paint House II, min/second-min optimization
16. Minimum Falling Path Sum II, min/second-min optimization
17. Bitset Subset Sum

- 18. Divide and Conquer DP optimization preview
 - 19. Knuth optimization preview
 - 20. Convex Hull Trick preview
-

1. What Is DP Optimization?

DP optimization means improving a DP solution after the recurrence is already known.

Example:

Basic recurrence:

```
dp[i] = min(dp[j] + cost(j, i)) for all j < i
```

Naive time:

```
O(n2)
```

If n is 100,000, this is too slow.

Optimization asks:

```
Can we compute this transition faster?  
Can we avoid trying every j?  
Can we reuse previous computation?  
Can we reduce memory?
```

2. Why DP Optimization Exists

Many DP solutions are easy to derive but too slow.

Example:

Longest Increasing Subsequence.

Basic DP:

```
dp[i] = 1 + max(dp[j]) where j < i and nums[j] < nums[i]
```

Naive:

```
O(n2)
```

Optimized:

```
O(n log n)
```

using binary search.

Another example:

Jump Game VI:

```
dp[i] = nums[i] + max(dp[j])  
where i - k <= j < i
```

Naive:

```
O(nk)
```

Optimized:

```
O(n)
```

using monotonic deque.

3. Common DP Optimization Types

Optimization	Main Idea	Example
Space optimization	Keep only needed previous states	Fibonacci
Rolling array	Keep previous row instead of full table	Unique Paths
Prefix sum	Replace repeated range sum with $O(1)$ query	Range DP
Difference array	Apply bulk transitions efficiently	DP range updates
Monotonic deque	Maintain max/min over sliding window	Jump Game VI
Binary search	Replace linear search in sorted structure	LIS
Bitset	Use machine-word parallelism	Subset Sum
Min/second-min	Avoid scanning all previous columns	Paint House II
State compression	Store only active state variables	Stock DP
Sparse memoization	Store only reachable states	Digit DP / graph DP
Divide and conquer DP	Optimize partition DP with monotonic opt	Advanced
Knuth optimization	Optimize interval-like partition DP	Advanced
Convex Hull Trick	Optimize linear transition functions	Advanced

4. When to Optimize DP

First ask:

4.1 Is the Basic DP Correct?

Do not optimize before the recurrence is correct.

4.2 What Are the Constraints?

If n is 500, $O(n^2)$ may be fine.

If n is 100,000, $O(n^2)$ is impossible.

4.3 What Is Repeated?

Optimization usually targets repeated work:

- repeated range maximum
- repeated range minimum

- repeated range sum
- repeated scan over previous states
- repeated table rows
- repeated subset checks

4.4 Is There Structure?

Examples:

- sliding window
 - sorted order
 - monotonicity
 - prefix sums
 - linear functions
 - limited dependency distance
-

5. When Not to Optimize

Do not optimize when:

5.1 Basic DP Already Passes

Unnecessary optimization can introduce bugs.

5.2 You Cannot Explain the Optimized State

In interviews, a clear $O(n^2)$ solution may be better than a broken $O(n \log n)$ solution.

5.3 The Optimization Requires Unproven Assumptions

Advanced optimizations like Knuth or divide-and-conquer DP require specific mathematical properties.

5.4 Code Complexity Is Too High

If the interviewer expects normal DP, do not overcomplicate.

6. Mental Model

DP optimization usually answers one question:

How can I compute the same transition faster?

Examples:

Need max over last k states?

Use deque.

Need sum over range?

Use prefix sum.

Need previous smaller value?

Use binary search or Fenwick tree.

Need only previous row?

Use rolling array.

Need subset sums?

Use bitset.

7. Pattern Recognition Checklist

Ask:

21. Does $dp[i]$ depend only on $dp[i - 1]$ and $dp[i - 2]$?
 22. Does $dp[row][col]$ depend only on previous row/current row?
 23. Does the recurrence repeatedly asks for range sum?
 24. Does it repeatedly asks for range max/min?
 25. Is the range sliding with fixed width k ?
 26. Is the transition over all $j < i$?
 27. Are states sorted by value?
 28. Can binary search replace scanning?
 29. Is it a knapsack/subset-sum problem?
 30. Can bitset help?
 31. Does the optimal split point move monotonically?
 32. Is there a linear expression like $m \cdot x + b$?
-

8. Optimization 1 — Simple Space Optimization

8.1 Idea

If a DP state only depends on the previous one or two states, do not store the full array.

Example:

Fibonacci:

```
dp[i] = dp[i - 1] + dp[i - 2]
```

Only need:

```
prev2  
prev1
```

8.2 Example — Fibonacci

```
class Solution {  
public:  
    int fib(int n) {  
        if (n <= 1) {  
            return n;  
        }  
  
        int prev2 = 0;  
        int prev1 = 1;  
  
        for (int i = 2; i <= n; i++) {  
            int curr = prev1 + prev2;  
            prev2 = prev1;  
            prev1 = curr;  
        }  
    }  
};
```

```
        return prev1;
    }
};
```

8.3 Complexity

Before:

```
Time: O(n)
Space: O(n)
```

After:

```
Time: O(n)
Space: O(1)
```

9. Optimization 2 — Rolling Array for Grid DP

9.1 Idea

If:

```
dp[r][c]
```

depends only on:

```
previous row
current row left cell
```

then one row is enough.

9.2 Example — Unique Paths

2D recurrence:

```
dp[r][c] = dp[r - 1][c] + dp[r][c - 1]
```

Space optimized:

```
class Solution {
public:
    int uniquePaths(int m, int n) {
        vector<int> dp(n, 1);

        for (int r = 1; r < m; r++) {
            for (int c = 1; c < n; c++) {
                dp[c] += dp[c - 1];
            }
        }

        return dp[n - 1];
    }
};
```

9.3 Why This Works

At cell c:

dp[c] before update = value from previous row
dp[c - 1] after update = value from current row left cell

So:

dp[c] = dp[c] + dp[c - 1];

means:

top + left

10. Optimization 3 — Rolling Array for Minimum Path Sum

10.1 Recurrence

dp[r][c] = grid[r][c] + min(dp[r - 1][c], dp[r][c - 1])

10.2 Optimized Code

```
class Solution {
public:
    int minPathSum(vector<vector<int>>& grid) {
        int rows = grid.size();
        int cols = grid[0].size();

        vector<int> dp(cols, 0);

        dp[0] = grid[0][0];

        for (int c = 1; c < cols; c++) {
            dp[c] = dp[c - 1] + grid[0][c];
        }

        for (int r = 1; r < rows; r++) {
            dp[0] += grid[r][0];

            for (int c = 1; c < cols; c++) {
                dp[c] = grid[r][c] + min(dp[c], dp[c - 1]);
            }

            return dp[cols - 1];
        }
    }
};
```

10.3 Important Lesson

In rolling array DP, update order matters.

If you update in the wrong direction, you may overwrite needed old values.

11. Optimization 4 — 0/1 Knapsack Space Optimization

11.1 Original 2D State

`dp[i][w]` = best value using first `i` items with capacity `w`

11.2 Optimized 1D State

`dp[w]` = best value using processed items with capacity `w`

11.3 Backward Loop Rule

For 0/1 knapsack:

```
for (int w = capacity; w >= weight; w--)
```

This prevents using the same item multiple times.

11.4 Code

```
int knapsack01(vector<int>& weights, vector<int>& values, int capacity) {
    int n = weights.size();

    vector<int> dp(capacity + 1, 0);

    for (int i = 0; i < n; i++) {
        for (int w = capacity; w >= weights[i]; w--) {
            dp[w] = max(dp[w], dp[w - weights[i]] + values[i]);
        }
    }

    return dp[capacity];
}
```

12. Optimization 5 — Prefix Sum Optimization

12.1 Idea

If DP repeatedly needs:

```
sum(l, r)
```

precompute prefix sums.

```
prefix[i + 1] = prefix[i] + nums[i]
```

Then:

```
sum(l, r) = prefix[r + 1] - prefix[l]
```

12.2 Example Use Case

Suppose:

```
dp[i] = min(dp[j] + sum(j, i - 1))
```

Without prefix sum, every sum costs $O(n)$.

With prefix sum, each sum costs $O(1)$.

13. Full Problem Walkthrough 1 — Prefix Sum Inside DP

13.1 Problem

Given nums, split prefix into groups. Cost of group $[j, i - 1]$ is sum of that group.

Define:

```
dp[i] = minimum cost to partition first i elements
```

Transition:

```
dp[i] = min(dp[j] + sum(j, i - 1))
```

13.2 Prefix Sum

```
vector<int> prefix(n + 1, 0);

for (int i = 0; i < n; i++) {
    prefix[i + 1] = prefix[i] + nums[i];
}
```

Then:

```
int groupSum = prefix[i] - prefix[j];
```

13.3 Lesson

Prefix sum does not always reduce $O(n^2)$ to $O(n)$, but it may reduce $O(n^3)$ to $O(n^2)$.

That is still a huge improvement.

14. Optimization 6 — Prefix DP / Cumulative Ways

14.1 Common Pattern

Suppose:

```
dp[i] = dp[i - 1] + dp[i - 2] + ... + dp[i - k]
```

Naive transition costs $O(k)$ per i .

Use a sliding sum:

```
windowSum = dp[i - 1] + ... + dp[i - k]
```

Then:

```
dp[i] = windowSum
```

Update window as i moves.

14.2 Example

Count ways to climb n stairs if you can jump 1 to k steps.

Naive:

```
for i:
    for jump = 1 to k:
        dp[i] += dp[i - jump]
```

Optimized:

```
class Solution {
public:
    long long countWays(int n, int k) {
        vector<long long> dp(n + 1, 0);

        dp[0] = 1;

        long long window = dp[0];

        for (int i = 1; i <= n; i++) {
            dp[i] = window;

            window += dp[i];

            if (i - k >= 0) {
                window -= dp[i - k];
            }
        }

        return dp[n];
    }
};
```

14.3 Complexity

Before:

$O(nk)$

After:

$O(n)$

15. Optimization 7 — Min / Second-Min Optimization

15.1 Pattern

Suppose:

```
dp[r][c] = cost[r][c] + min(dp[r - 1][pc])  
where pc != c
```

Naive:

For every cell, scan all previous columns.
 $O(n * m^2)$

Optimization:

Track the smallest and second smallest values in previous row.

If current column is not the min column, use min.

If current column is the min column, use second min.

16. Full Problem Walkthrough 2 — Paint House II

16.1 Problem Summary

There are n houses and k colors.

Adjacent houses cannot have same color.

Return minimum cost.

16.2 Recurrence

```
dp[i][color] = costs[i][color] + min(dp[i - 1][otherColor])
```

where:

```
otherColor != color
```

16.3 Naive Complexity

$O(n * k^2)$

16.4 Optimized Code

```
class Solution {  
public:  
    int minCostII(vector<vector<int>>& costs) {  
        if (costs.empty()) {  
            return 0;  
        }  
  
        int n = costs.size();  
        int k = costs[0].size();  
  
        vector<int> prev = costs[0];  
  
        for (int i = 1; i < n; i++) {  
            int min1 = -1;  
            int min2 = -1;  
  
            for (int c = 0; c < k; c++) {  
                if (min1 == -1 || prev[c] < prev[min1]) {
```

```

        min2 = min1;
        min1 = c;
    } else if (min2 == -1 || prev[c] < prev[min2]) {
        min2 = c;
    }
}

vector<int> curr(k, 0);

for (int c = 0; c < k; c++) {
    int bestPrev = (c == min1) ? prev[min2] :
prev[min1];
    curr[c] = costs[i][c] + bestPrev;
}

prev = curr;
}

return *min_element(prev.begin(), prev.end());
}
};

```

16.5 Complexity

Before:

$O(n * k^2)$

After:

$O(n * k)$

17. Full Problem Walkthrough 3 — Minimum Falling Path Sum II

This uses the same min/second-min optimization.

17.1 Recurrence

$dp[r][c] = grid[r][c] + \min(dp[r - 1][pc])$
 where $pc \neq c$

17.2 Code

```

class Solution {
public:
    int minFallingPathSum(vector<vector<int>>& grid) {
        int n = grid.size();

        vector<int> prev = grid[0];

        for (int r = 1; r < n; r++) {

```

```

        int min1 = -1;
        int min2 = -1;

        for (int c = 0; c < n; c++) {
            if (min1 == -1 || prev[c] < prev[min1]) {
                min2 = min1;
                min1 = c;
            } else if (min2 == -1 || prev[c] < prev[min2]) {
                min2 = c;
            }
        }

        vector<int> curr(n);

        for (int c = 0; c < n; c++) {
            int bestPrev = (c == min1) ? prev[min2] :
prev[min1];
            curr[c] = grid[r][c] + bestPrev;
        }

        prev = curr;
    }

    return *min_element(prev.begin(), prev.end());
};

```

18. Optimization 8 — Monotonic Deque Optimization

18.1 Pattern

Use this when recurrence needs max/min over a sliding window.

Example:

```

dp[i] = nums[i] + max(dp[j])
where i - k <= j < i

```

Naive:

For every i, scan last k states.
 $O(nk)$

Optimized:

Maintain a deque of candidate indices with decreasing dp values.
 Then max is always at front.

19. How Monotonic Deque Works

For max query:

- Deque stores indices.
- dp values in deque are decreasing.
- Front has maximum dp.
- Remove indices outside window.
- Remove worse values from back.

Pseudo:

```
while (!dq.empty() && dq.front() < i - k) {
    dq.pop_front();
}

dp[i] = nums[i] + dp[dq.front()];

while (!dq.empty() && dp[dq.back()] <= dp[i]) {
    dq.pop_back();
}

dq.push_back(i);
```

20. Full Problem Walkthrough 4 — LeetCode 1696: Jump Game VI

20.1 Problem Summary

Given nums and k.

Start at index 0.

From index i, you can jump to any index:

$i + 1$ to $i + k$

Return maximum score to reach last index.

Score is sum of visited nums.

20.2 DP Recurrence

$dp[i] = nums[i] + \max(dp[j])$
where $i - k \leq j < i$

20.3 Naive

$O(nk)$

20.4 Optimized With Deque

```
class Solution {
public:
    int maxResult(vector<int>& nums, int k) {
        int n = nums.size();

        vector<int> dp(n, 0);
        dp[0] = nums[0];
```

```

deque<int> dq;
dq.push_back(0);

for (int i = 1; i < n; i++) {
    while (!dq.empty() && dq.front() < i - k) {
        dq.pop_front();
    }

    dp[i] = nums[i] + dp[dq.front()];

    while (!dq.empty() && dp[dq.back()] <= dp[i]) {
        dq.pop_back();
    }

    dq.push_back(i);
}

return dp[n - 1];
};

```

20.5 Complexity

Before:

$O(nk)$

After:

$O(n)$

because each index enters and leaves deque once.

21. Full Problem Walkthrough 5 — LeetCode 1425: Constrained Subsequence Sum

21.1 Problem Summary

Find maximum sum of a non-empty subsequence such that adjacent chosen indices differ by at most k .

21.2 DP Recurrence

```

dp[i] = nums[i] + max(0, max(dp[j]))
where i - k <= j < i

```

Why max with 0?

If all previous choices hurt, start new subsequence at i .

21.3 Optimized Code

```

class Solution {
public:

```

```

int constrainedSubsetSum(vector<int>& nums, int k) {
    int n = nums.size();

    vector<int> dp(n, 0);
    deque<int> dq;

    int answer = nums[0];

    for (int i = 0; i < n; i++) {
        while (!dq.empty() && dq.front() < i - k) {
            dq.pop_front();
        }

        dp[i] = nums[i];

        if (!dq.empty()) {
            dp[i] += max(0, dp[dq.front()]);
        }

        answer = max(answer, dp[i]);

        while (!dq.empty() && dp[dq.back()] <= dp[i]) {
            dq.pop_back();
        }

        dq.push_back(i);
    }

    return answer;
};

```

21.4 Complexity

Time: $O(n)$
 Space: $O(n)$

Space can be optimized if only dp values in deque are needed, but array is clearer.

22. Optimization 9 — Prefix Sum + Monotonic Deque

22.1 Pattern

Sometimes recurrence or problem uses prefix sums and asks for shortest/longest subarray with a constraint.

This is not always called DP, but it is closely related to optimizing range-based states. Classic problem:

Shortest subarray with sum at least K

Need:

$$\text{prefix}[j] - \text{prefix}[i] \geq K$$

Minimize:

$$j - i$$

Use deque of candidate prefix indices.

23. Full Problem Walkthrough 6 — LeetCode 862: Shortest Subarray With Sum at Least K

23.1 Problem Summary

Given nums possibly containing negatives, return length of shortest non-empty subarray with sum at least k.

23.2 Why Sliding Window Fails

Sliding window works only when all numbers are non-negative.

Here nums can contain negatives, so window sum is not monotonic.

23.3 Prefix Sum

$$\text{prefix}[i] = \text{sum of } \text{nums}[0..i-1]$$

Need smallest:

$$j - i$$

where:

$$\text{prefix}[j] - \text{prefix}[i] \geq k$$

23.4 Monotonic Deque Idea

Maintain indices with increasing prefix values.

If a later prefix is smaller than an earlier prefix, earlier one is worse.

23.5 Code

```
class Solution {
public:
    int shortestSubarray(vector<int>& nums, int k) {
        int n = nums.size();

        vector<long long> prefix(n + 1, 0);

        for (int i = 0; i < n; i++) {
            prefix[i + 1] = prefix[i] + nums[i];
        }

        deque<int> dq;
        int answer = n + 1;
```

```

        for (int j = 0; j <= n; j++) {
            while (!dq.empty() && prefix[j] - prefix[dq.front()] >=
k) {
                answer = min(answer, j - dq.front());
                dq.pop_front();
            }

            while (!dq.empty() && prefix[dq.back()] >= prefix[j]) {
                dq.pop_back();
            }

            dq.push_back(j);
        }

        return answer == n + 1 ? -1 : answer;
    }
};

```

23.6 Complexity

Time: $O(n)$
 Space: $O(n)$

24. Optimization 10 — LIS Binary Search Optimization

24.1 Basic DP

$dp[i]$ = length of LIS ending at i

Naive:

$O(n^2)$

24.2 Optimized Idea

Maintain:

$tails[len]$ = smallest possible ending value of an increasing subsequence of length $len + 1$

For each x :

- Find first tail $\geq x$.
- Replace it with x .
- If none, append x .

24.3 Why It Works

Smaller ending values are better because they leave more room for future elements.

Example:

For subsequence length 3:

ending at 5 is better than ending at 9

because more future numbers can extend 5.

25. Full Problem Walkthrough 7 — LeetCode 300: Longest Increasing Subsequence

```
class Solution {
public:
    int lengthOfLIS(vector<int>& nums) {
        vector<int> tails;

        for (int x : nums) {
            auto it = lower_bound(tails.begin(), tails.end(), x);

            if (it == tails.end()) {
                tails.push_back(x);
            } else {
                *it = x;
            }
        }

        return tails.size();
    }
};
```

25.1 Strict vs Non-Decreasing

For strictly increasing LIS:

lower_bound

For non-decreasing subsequence:

upper_bound

25.2 Complexity

Time: $O(n \log n)$

Space: $O(n)$

26. Optimization 11 — Bitset Optimization for Subset Sum

26.1 Pattern

Subset sum:

$dp[s] = \text{true}$ if sum s is possible

Normal 0/1 DP:

```
O(n * target)
```

Bitset optimization uses bit operations to update many sums at once.

Core idea:

```
bits |= bits << x;
```

If sum s was possible before, sum $s + x$ becomes possible.

26.2 Example

Initially:

```
bits[0] = 1
```

For $x = 3$:

```
bits << 3
```

makes sum 3 possible.

For $x = 5$:

previous possible sums 0 and 3 create 5 and 8.

27. Full Problem Walkthrough 8 — Partition Equal Subset Sum With Bitset

```
class Solution {
public:
    bool canPartition(vector<int>& nums) {
        int total = accumulate(nums.begin(), nums.end(), 0);

        if (total % 2 == 1) {
            return false;
        }

        int target = total / 2;

        bitset<20001> bits;
        bits[0] = 1;

        for (int x : nums) {
            bits |= bits << x;
        }

        return bits[target];
    }
};
```

27.1 Complexity

Conceptually:

```
O(n * target / word_size)
```

Very fast in practice.

27.2 Limitation

C++ `bitset` size must be known at compile time.

If target is dynamic and large, use custom dynamic `bitset` or normal DP.

28. Optimization 12 — State Compression

28.1 Idea

If DP has several named states per index, store only current values.

Example:

Stock with cooldown:

```
hold
sold
rest
```

Instead of:

```
hold[i], sold[i], rest[i]
```

use variables.

28.2 Code

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int hold = -prices[0];
        int sold = 0;
        int rest = 0;

        for (int i = 1; i < prices.size(); i++) {
            int prevHold = hold;
            int prevSold = sold;
            int prevRest = rest;

            hold = max(prevHold, prevRest - prices[i]);
            sold = prevHold + prices[i];
            rest = max(prevRest, prevSold);
        }

        return max(sold, rest);
    }
};
```

28.3 Lesson

State compression is safe only if you preserve old values before overwriting.

29. Optimization 13 — Sparse Memoization

29.1 Idea

Sometimes the theoretical DP table is huge, but only a small number of states are reachable.

Use:

```
unordered_map  
map  
memoized DFS
```

Examples:

- Shopping Offers
- Digit DP with complex state
- Graph state DP
- recursive game DP
- vector state DP

29.2 Example

```
map<vector<int>, int> memo;
```

Used for multidimensional needs in Shopping Offers.

29.3 Tradeoff

Sparse memoization saves memory but map/unordered_map has overhead.

Use it when dense arrays are too large or inconvenient.

30. Optimization 14 — Divide and Conquer DP Optimization Preview

30.1 Pattern

This applies to DP like:

```
dp[g][i] = min(dp[g - 1][k] + cost(k, i))  
where k < i
```

Naive:

```
O(groups * n2)
```

Optimized if optimal k is monotonic:

```
opt[g][i] <= opt[g][i + 1]
```

Then compute one row with divide and conquer.

30.2 Requirement

The optimization is valid only when the best split point moves monotonically.

Do not assume it blindly.

30.3 Conceptual Function

```
compute(g, left, right, optLeft, optRight)
```

It computes $dp[g][mid]$ by searching k only in:

```
[optLeft, optRight]
```

Then recursively computes left and right halves with narrowed opt ranges.

30.4 Where It Appears

- partition array into k groups
- clustering DP
- some cost functions with quadrangle inequality

This is advanced and usually not required in standard interviews, but useful for DE Shaw-level harder problems.

31. Optimization 15 — Knuth Optimization Preview

31.1 Pattern

Applies to interval/partition DP:

```
dp[l][r] = min(dp[l][k] + dp[k][r] + cost(l, r))
```

If optimal split satisfies:

```
opt[l][r - 1] <= opt[l][r] <= opt[l + 1][r]
```

Then k search range can be reduced.

31.2 Complexity Improvement

Naive interval DP:

```
 $O(n^3)$ 
```

Knuth optimized:

```
 $O(n^2)$ 
```

31.3 Common Examples

- Optimal Binary Search Tree
- certain merge cost DP problems
- some file merging variants

31.4 Warning

Knuth optimization requires strict mathematical conditions.

Do not apply unless you know they hold.

32. Optimization 16 — Convex Hull Trick Preview

32.1 Pattern

Applies to recurrence:

```
dp[i] = min(dp[j] + m[j] * x[i] + b[j])
```

Each previous j defines a line:

```
y = m[j] * x + b[j]
```

For each i, query minimum value at x[i].

32.2 Goal

Instead of scanning all j:

```
O(n2)
```

use a structure to query best line:

```
O(log n)
```

or amortized:

```
O(1)
```

depending on monotonicity.

32.3 Where It Appears

- dynamic programming with linear cost
- transportation cost problems
- partition DP
- quadratic-looking transitions after expansion

32.4 Interview Note

Convex Hull Trick is advanced. Usually, recognizing that the transition is linear is enough unless the role is very algorithm-heavy.

33. Full Problem Walkthrough 9 — Count Vowels Permutation With State Compression

33.1 Problem Summary

Count strings of length n made from vowels with transition rules.

Each vowel can follow certain previous vowels.

33.2 State

a, e, i, o, u = number of strings ending in that vowel

33.3 Code

```
class Solution {
public:
    int countVowelPermutation(int n) {
        const int MOD = 1000000007;

        long long a = 1;
        long long e = 1;
        long long i = 1;
```

```

long long o = 1;
long long u = 1;

for (int len = 2; len <= n; len++) {
    long long na = (e + i + u) % MOD;
    long long ne = (a + i) % MOD;
    long long ni = (e + o) % MOD;
    long long no = i;
    long long nu = (i + o) % MOD;

    a = na;
    e = ne;
    i = ni;
    o = no;
    u = nu;
}

return (a + e + i + o + u) % MOD;
}
};

```

33.4 Lesson

When number of states is fixed and small, named variables are often cleaner than arrays.

34. Full Problem Walkthrough 10 — Maximal Square Space Optimized

34.1 Recurrence

```
dp[r][c] = 1 + min(top, left, diagonal)
```

Need top-left diagonal from previous row.

When using 1D array, store diagonal in variable.

34.2 Code

```

class Solution {
public:
    int maximalSquare(vector<vector<char>>& matrix) {
        int rows = matrix.size();
        int cols = matrix[0].size();

        vector<int> dp(cols + 1, 0);

        int best = 0;

        for (int r = 1; r <= rows; r++) {
            int prevDiagonal = 0;

```

```

        for (int c = 1; c <= cols; c++) {
            int temp = dp[c];

            if (matrix[r - 1][c - 1] == '1') {
                dp[c] = 1 + min({dp[c], dp[c - 1],
prevDiagonal});
                best = max(best, dp[c]);
            } else {
                dp[c] = 0;
            }

            prevDiagonal = temp;
        }

        return best * best;
    }
};

```

34.3 Important Lesson

For 2D-to-1D compression, sometimes you need an extra variable for diagonal dependency.

35. Common DP Optimization Edge Cases

35.1 Overwriting Needed Values

Rolling array bugs often come from overwriting a state too early.

35.2 Wrong Loop Direction

Common in knapsack:

- 0/1 uses backward
- unbounded uses forward

35.3 Empty Deque

Always ensure deque has valid candidates before accessing front.

35.4 Window Expiry

For sliding window DP:

```
while dq.front() < i - k
```

must be correct.

35.5 Strict vs Non-Strict Binary Search

LIS:

- strictly increasing: lower_bound
- non-decreasing: upper_bound

35.6 Advanced Optimization Misuse

Do not use Knuth or divide-and-conquer optimization without verifying conditions.

36. Common DP Optimization Mistakes

Mistake 1 — Optimizing Before Deriving DP

Always derive the recurrence first.

Mistake 2 — Using Rolling Array With Wrong Direction

If current row depends on previous row and current row, direction matters.

Mistake 3 — Monotonic Deque Stores Values Instead of Indices

Usually store indices so you can remove expired elements.

Mistake 4 — Forgetting to Remove Expired Indices

The max/min may come from outside the valid window.

Mistake 5 — Misusing Bitset for Negative Values

Bitset subset sum works naturally for non-negative sums.

Negative values need offset or a different approach.

Mistake 6 — Assuming LIS Tails Is the Actual Sequence

tails gives length, not necessarily the real subsequence unless additional parent tracking is used.

37. Interview Explanation Templates

37.1 Space Optimization

The recurrence only depends on the previous one or two states, so I do not need to store the entire DP array.

37.2 Rolling Array

Each row depends only on the previous row and current row values, so I compress the 2D table into one row.

37.3 Prefix Sum Optimization

The recurrence repeatedly needs range sums, so I precompute prefix sums to answer each range query in $O(1)$.

37.4 Monotonic Deque DP

The transition needs the maximum DP value over a sliding window. I maintain candidate indices in decreasing DP order so the best value is always at the front.

37.5 LIS Binary Search

Instead of storing the best subsequence ending at each index, I store the smallest possible tail for each length. Binary search updates the first tail that is greater than or equal to the current value.

37.6 Bitset Optimization

The subset-sum transition can be represented as bit shifts. Shifting the bitset by x marks all sums reachable after adding x .

38. Complexity Improvement Table

Pattern	Naive	Optimized
Fibonacci space	$O(n)$ space	$O(1)$ space
Grid DP space	$O(mn)$ space	$O(n)$ space
0/1 Knapsack space	$O(nW)$ space	$O(W)$ space
Range sum in DP	$O(n^3)$	$O(n^2)$
Sliding sum DP	$O(nk)$	$O(n)$
Jump Game VI	$O(nk)$	$O(n)$
LIS	$O(n^2)$	$O(n \log n)$
Paint House II	$O(nk^2)$	$O(nk)$
Subset Sum	$O(nW)$	$O(nW / \text{word_size})$
Divide and conquer DP	$O(kn^2)$	$O(kn \log n)$ or $O(kn)$ depending form
Knuth optimization	$O(n^3)$	$O(n^2)$
Convex Hull Trick	$O(n^2)$	$O(n \log n)$ or $O(n)$

39. Expanded Practice Set

39.1 Must-Do Medium

- 33. Unique Paths space optimized
- 34. Minimum Path Sum space optimized
- 35. Maximal Square space optimized
- 36. Partition Equal Subset Sum bitset
- 37. Longest Increasing Subsequence $O(n \log n)$
- 38. Paint House II
- 39. Minimum Falling Path Sum II
- 40. Count Vowels Permutation
- 41. Coin Change space optimized

39.2 Strong Interview Practice

- 42. Jump Game VI
- 43. Constrained Subsequence Sum
- 44. Shortest Subarray With Sum at Least K
- 45. Minimum Cost For Tickets optimized
- 46. Number of Ways to Stay in Same Place After Some Steps
- 47. Count Number of Teams optimized variants
- 48. Russian Doll Envelopes
- 49. Largest Divisible Subset
- 50. Maximum Sum of 3 Non-Overlapping Subarrays

39.3 Hard / Stretch

- 51. Divide and Conquer DP examples
 - 52. Knuth Optimization examples
 - 53. Convex Hull Trick DP examples
 - 54. Alien trick / Lagrangian optimization preview
 - 55. Dynamic bitset subset sum
 - 56. DP with monotonic queue optimization hard variants
-

40. Suggested Study Order

Step 1 — Space Optimization

Solve:

- 57. Fibonacci
- 58. Unique Paths
- 59. Minimum Path Sum
- 60. Maximal Square

Goal:

Learn safe table compression.

Step 2 — Knapsack Compression

Solve:

- 61. Partition Equal Subset Sum
- 62. Coin Change
- 63. Coin Change II

Goal:

Master loop direction.

Step 3 — Prefix Optimization

Solve:

- 64. Range-sum DP examples
- 65. ways with last k jumps
- 66. Minimum Cost For Tickets

Goal:

Replace repeated sums/scans.

Step 4 — Monotonic Deque DP

Solve:

- 67. Jump Game VI
- 68. Constrained Subsequence Sum
- 69. Shortest Subarray With Sum at Least K

Goal:

Master sliding max/min.

Step 5 — Binary Search DP

Solve:

- 70. LIS
- 71. Russian Doll Envelopes

Goal:

Recognize sorted-tail optimization.

Step 6 — Advanced Preview

Study:

- 72. Divide and Conquer DP

73. Knuth optimization

74. Convex Hull Trick

Goal:

Recognize advanced transition structures.

41. Pattern Decision Table

Problem Clue	Optimization
only previous 1 or 2 states	$O(1)$ space
grid depends on top/left	rolling array
0/1 knapsack	1D backward loop
unbounded knapsack	1D forward loop
repeated range sum	prefix sum
sum of last k dp values	sliding window sum
max/min over last k states	monotonic deque
previous smaller/larger value	binary search / Fenwick
choose color different from previous	min/second-min
subset sum	bitset
sparse reachable states	memo map
transition has monotonic optimal split	divide and conquer DP
interval split opt is monotonic	Knuth optimization
transition is min of lines	Convex Hull Trick

42. Mini Problem 1 — Sliding Window Ways

Count ways to reach n if from step i you can jump 1 to k steps.

```
long long countWays(int n, int k) {  
    vector<long long> dp(n + 1, 0);  
  
    dp[0] = 1;  
  
    long long window = dp[0];
```

```

    for (int i = 1; i <= n; i++) {
        dp[i] = window;

        window += dp[i];

        if (i - k >= 0) {
            window -= dp[i - k];
        }
    }

    return dp[n];
}

```

43. Mini Problem 2 — Maximum Sum With Last K Dependency

```

int maxScore(vector<int>& nums, int k) {
    int n = nums.size();

    vector<int> dp(n, 0);
    deque<int> dq;

    dp[0] = nums[0];
    dq.push_back(0);

    for (int i = 1; i < n; i++) {
        while (!dq.empty() && dq.front() < i - k) {
            dq.pop_front();
        }

        dp[i] = nums[i] + dp[dq.front()];

        while (!dq.empty() && dp[dq.back()] <= dp[i]) {
            dq.pop_back();
        }

        dq.push_back(i);
    }

    return dp[n - 1];
}

```

44. Mini Problem 3 — Dynamic Programming With Prefix Minimum

Suppose:

```
dp[i] = cost[i] + min(dp[0], dp[1], ..., dp[i - 1])
```

Naive:

```
O(n2)
```

Optimized:

```
int best = dp[0];

for (int i = 1; i < n; i++) {
    dp[i] = cost[i] + best;
    best = min(best, dp[i]);
}
```

Lesson:

If transition needs all previous minimum, maintain running minimum.

45. Mini Problem 4 — Running Maximum DP

Suppose:

```
dp[i] = nums[i] + max(dp[j])
for all j < i
```

Use:

```
int best = dp[0];

for (int i = 1; i < n; i++) {
    dp[i] = nums[i] + best;
    best = max(best, dp[i]);
}
```

If j is limited to a window, use deque instead.

46. Debugging DP Optimization

If optimized DP fails, check:

- 75. Is the original recurrence correct?
 - 76. Did optimization preserve the same recurrence?
 - 77. Are you overwriting states too early?
 - 78. Is loop direction correct?
 - 79. Are window boundaries correct?
 - 80. Does deque store indices, not just values?
 - 81. Are expired indices removed before querying?
 - 82. Is binary search using lower_bound or upper_bound correctly?
 - 83. Did you reset temporary arrays each row?
 - 84. Are advanced optimization assumptions valid?
-

47. What to Master Before Chapter 44

Before moving to String Algorithms, make sure you can:

- Space optimize Fibonacci-style DP
 - Space optimize grid DP
 - Space optimize knapsack
 - Explain backward vs forward knapsack loops
 - Use prefix sums inside DP
 - Use sliding window sums for DP
 - Use monotonic deque for max/min DP transitions
 - Solve Jump Game VI
 - Solve Constrained Subsequence Sum
 - Solve LIS in $O(n \log n)$
 - Use min/second-min optimization
 - Understand bitset subset sum
 - Recognize advanced DP optimization structures conceptually
-

48. Final Cheat Sheet

[O\(1\) Space Fibonacci](#)

```
curr = prev1 + prev2;  
prev2 = prev1;  
prev1 = curr;
```

[Rolling Grid](#)

```
dp[c] = dp[c] + dp[c - 1];
```

[0/1 Knapsack](#)

```
for (int s = target; s >= x; s--) {  
    dp[s] = max(dp[s], dp[s - x] + value);  
}
```

[Unbounded Knapsack](#)

```
for (int s = x; s <= target; s++) {  
    dp[s] = min(dp[s], dp[s - x] + 1);  
}
```

[Prefix Sum](#)

```
sum(l, r) = prefix[r + 1] - prefix[l];
```

[Sliding Window Sum DP](#)

```
dp[i] = window;  
window += dp[i];  
window -= dp[i - k];
```

[Monotonic Deque Max](#)

```

while (!dq.empty() && dq.front() < i - k) {
    dq.pop_front();
}

dp[i] = nums[i] + dp[dq.front()];

while (!dq.empty() && dp[dq.back()] <= dp[i]) {
    dq.pop_back();
}

dq.push_back(i);

```

LIS

```

auto it = lower_bound(tails.begin(), tails.end(), x);

if (it == tails.end()) {
    tails.push_back(x);
} else {
    *it = x;
}

```

Bitset Subset Sum

```
bits |= bits << x;
```

49. Core Takeaway

DP optimization is not a separate DP type.

It is the process of making a correct DP faster or smaller.

The workflow is:

1. Define the normal DP.
2. Check constraints.
3. Identify repeated work.
4. Find structure.
5. Apply the right optimization.

Most useful interview-level optimizations are:

- rolling array
- prefix sum
- monotonic deque
- binary search
- min/second-min
- bitset
- state compression

Advanced optimizations like:

- divide and conquer DP
- Knuth optimization
- Convex Hull Trick

are powerful but require special conditions.

For DE Shaw-level preparation, you should at least recognize them and deeply master the practical ones: rolling arrays, prefix sums, monotonic deque, LIS binary search, and bitset optimization.
